**Berufsmaturitätsschule Zürich**
Ausrichtung: Technik, Architektur, Life Sciences

# Berufsmaturitätsarbeit (BMA)

Abschlussprojekt gemäss Art. 11 Abs. 4 BMV

# Emulation mit Ayn Thor

Dekonstruktion von Software-Emulatoren und empirische Grenzwertanalyse auf moderner Handheld-Hardware

**Titelbild: Ayn Thor im Benchmark-Betrieb**

*Foto des Ayn Thor mit sichtbaren Scene-5-Echtzeit-Telemetriedaten*

[Dateiablage: abbildungen/titelbild.jpg]

| | |
|---|---|
| **Oberthema** | Was läuft im Hintergrund? |
| **Titel der Arbeit** | Emulation mit Ayn Thor |
| **Verfasser** | David Martin Felicio Reichlin<br>Gabriel Di Giuseppe |
| **Klasse** | BIN23e |
| **Schule** | Berufsmaturitätsschule Zürich (BMZ) |
| **Begleitende Lehrperson** | Alexander Körner |
| **Abgabedatum** | 27. November 2026 |
| **Version** | 2.0 (Gemäss BMZ-Musterlayout 2026) |

# 1. Abstract

Diese Berufsmaturitätsarbeit untersucht die Funktionsweise von Software-Emulation auf Systemebene und analysiert die praktischen Leistungsgrenzen auf modernen Android-basierten Gaming-Handhelds. Im Zentrum der Untersuchung steht die Frage, welcher mathematische, architektonische und rechentechnische Aufwand erbracht werden muss, um fremde Befehlssatzarchitekturen in Echtzeit auf einem Host-System zu übersetzen und welche konkreten Hardware-Flaschenhälse die Emulierbarkeit limitieren.

Als Untersuchungsmethodik dient einerseits eine systematische Fachliteraturrecherche zur theoretischen Fundierung der Rechnerarchitektur (Von-Neumann-Modell, Fetch-Decode-Execute-Zyklus, Finite State Machine, Interpreter vs. JIT-Compiler). Zur praktischen Veranschaulichung der theoretischen Prinzipien wird ein eigener modularer Referenz-Interpreter für die CHIP-8-Architektur in der Programmiersprache Rust entwickelt. Andererseits wird eine empirische Benchmark-Reihe mit standardisierter Telemetrie-Datenerfassung via Shizuku und Scene 5 auf dem Ayn Thor durchgeführt, bei der fünf Entwicklungsstufen (CHIP-8, Atari 2600, Nintendo DS, Sony PlayStation 2 und Nintendo Switch) hinsichtlich Ladezeiten, Bildstabilität, CPU-/GPU-Auslastung, Leistungsaufnahme und thermischer Grenzen quantitativ und qualitativ verglichen werden.

Die empirischen Messergebnisse belegen einen stark nicht-linearen, exponentiellen Anstieg des Ressourcen- und Energiebedarfs mit zunehmender Architekturkomplexität. Während einfache 8-Bit- und 32-Bit-Systeme (CHIP-8, Atari, NDS) auf dem Ayn Thor weniger als 15 % CPU-Auslastung und unter 3.5 W Systemleistung erfordern, führen hochgradig asynchrone Multi-Chip-Architekturen wie die PlayStation 2 sowie moderne JIT-Kompilierungen bei der Nintendo Switch zu Lastspitzen von über 90 % und einer Leistungsaufnahme von bis zu 10.4 W. Die Arbeit zeigt auf, dass nicht allein die reine Prozessortaktung, sondern insbesondere die Zyklensynchronisation zwischen Coprozessoren und der Treiber-Overhead mobiler Grafik-APIs die primären Limitierungsfaktoren mobiler Emulation darstellen.

# Inhaltsverzeichnis

**1. Abstract** **2**

**2. Einleitung** **5**
2.1. Thematischer Kontext & Bezug zum Oberthema . . . . . 5
2.2. Ziel der Arbeit . . . . . 5
2.3. Persönlicher Bezug & Motivation . . . . . 6
2.4. Forschungsfragen und wissenschaftliche Hypothesen . . . . . 6
2.5. Abgrenzung der Arbeit . . . . . 7
2.6. Methodischer Ansatz & Aufbau der Arbeit . . . . . 7

**3. Theoretische Grundlagen der Emulation** **9**
3.1. Statische Grundlagen eines Rechnersystems . . . . . 9
3.2. Definition & Differenzierung: Simulation, Emulation, Virtualisierung . . 10
3.3. Technische Kernkomponenten eines Emulators . . . . . 11
3.4. Funktionsweise im Hintergrund: Interpreter vs. JIT-Compiler . . . . . 11
3.5. Systemgrenzen und Flaschenhälse . . . . . 12

**4. Praktische Untersuchung und Messungen** **13**
4.1. Eigene Referenz-Implementation: CHIP-8 in Rust . . . . . 13
4.1.1. Modulare Softwarearchitektur (Core vs. Frontends) . . . . . 13
4.1.2. Abbildung der Zustandsmaschine im Speicher . . . . . 13
4.1.3. Implementierung des Fetch-Decode-Execute-Zyklus & Status-Flags 14
4.2. Testplattform Ayn Thor . . . . . 15
4.3. Mess-Setup und Telemetrie-Pipeline . . . . . 15
4.4. Auswahl der Testsysteme und Emulatoren . . . . . 17

**5. Auswertung und Diskussion** **18**
5.1. Tabellarischer Gesamtvergleich der Telemetriedaten . . . . . 18
5.2. Performance- und Framerate-Analyse . . . . . 18
5.3. Ressourcenbedarf, Energieeffizienz und Thermik . . . . . 20
5.4. Analyse der Grafik-Backends und Skalierung . . . . . 20
5.5. Überprüfung der wissenschaftlichen Hypothesen . . . . . 20
5.6. Technische und rechtliche Grenzen der Emulation . . . . . 22

**6. Schlusswort** **24**
6.1. Beantwortung der Leitfrage . . . . . 24
6.2. Wissenschaftliche Kernaussagen . . . . . 24
6.3. Kritische Würdigung und Ausblick . . . . . 25

**7. Quellenverzeichnis** **26**
7.1. Literaturverzeichnis . . . . . 26
7.2. Abbildungsverzeichnis . . . . . 26
7.3. Tabellenverzeichnis . . . . . 27

**8. Danksagung** **28**

**9. Bescheinigung** **29**

**A. Anhang** **30**
A.1. Deklaration zur Verwendung generativer KI-Systeme . . . . . 30
A.2. Protokollvorlage für Einzelmessungen (Scene 5) . . . . . 31

# 2. Einleitung

## 2.1. Thematischer Kontext & Bezug zum Oberthema

In der modernen Informationsgesellschaft unterliegt Computer-Hardware einem stetigen und rasanten Innovationszyklus. Während sich Prozessorgeschwindigkeiten, Speicherbandbreiten und Grafikleistungen alle paar Jahre vervielfachen, führt dies auf der Kehrseite zu einem gravierenden Problem: der programmierten und physikalischen Obsoleszenz älterer Rechnersysteme. Historische Videospiele, interaktive Medien und Spezialprogramme der vergangenen Jahrzehnte stellen heute unbestritten schützenswerte digitale Kulturgüter dar. Ihre Ausführung ist jedoch traditionell an spezifische, nicht mehr hergestellte Hardwareplattformen gekoppelt. Alternde Bauteile wie austrocknende Elektrolytkondensatoren, nachlassende Laseroptiken und korrodierende Leiterbahnen sorgen dafür, dass originale Konsolen zunehmend unzuverlässig oder gänzlich unbrauchbar werden.

Hier setzt die Technologie der Software-Emulation an. Ein Emulator rekonstruiert eine fremde oder historische Rechnerarchitektur vollständig innerhalb eines modernen Betriebssystems. Dadurch wird Software von ihrer ursprünglichen Hardware entkoppelt und für zukünftige Generationen archiviert und zugänglich gehalten. Gleichzeitig markiert die Emulation ein hochgradig anspruchsvolles Teilgebiet der angewandten Informatik: Sie verlangt ein tiefes Verständnis von Mikroarchitekturen, Maschinencode, Befehlssatzübersetzungen und synchronen Bus-Systemen. In der professionellen Softwareentwicklung und im Ingenieurwesen sind verwandte Prinzipien – wie etwa Virtualisierung und Instruction-Set-Translation – fundamental, beispielsweise bei der Entwicklung von Embedded Systems oder der Ausführung von x86-Legacy-Software auf modernen ARM-Servern. Diese Arbeit bettet die Emulationsthematik daher direkt in das BMA-Oberthema «Was läuft im Hintergrund?» ein.

## 2.2. Ziel der Arbeit

Das primäre Ziel dieser Berufsmaturitätsarbeit besteht darin, die Funktionsweise von Emulationssoftware auf Systemebene fundiert zu dekonstruieren und ihre praktischen Leistungsgrenzen auf moderner, mobiler Hardware empirisch zu erforschen. Im Zentrum steht dabei die Frage, welcher mathematische und architektonische Aufwand betrieben werden muss, damit Instruktionen einer völlig fremden Prozessorarchitektur auf einem modernen Host-System in Echtzeit verarbeitet werden können.

Anhand des Gaming-Handhelds *Ayn Thor* soll praxisnah untersucht werden, wie sich unterschiedliche Komplexitätsgrade emulierter Systeme – von einfachen 8-Bit-Architekturen bis hin zu modernen, hochparallelen Plattformen – auf Systemressourcen, Energieeffizienz und Ausführungsstabilität auswirken. Die Arbeit verfolgt das Ziel,

die oft als «Black Box» wahrgenommene Emulationstechnologie technisch transparent darzulegen und die konkreten Hardware- und Software-Flaschenhälse greifbar zu machen.

## 2.3. Persönlicher Bezug & Motivation

Als angehende Informatiker (Klasse BIN23e an der Berufsmaturitätsschule Zürich) verbindet uns beide ein ausgeprägtes Interesse an hardwarenaher Programmierung, Systemarchitekturen sowie der praktischen Optimierung von Algorithmen. Parallel dazu verfolgen wir seit Jahren die rasante Entwicklung im Bereich portabler Handheld-Hardware. Die Vorstellung, dass ein kompaktes mobiles Gerät in der Lage ist, hochentwickelte Heimkonsolen wie die PlayStation 2 oder gar moderne Systeme softwareseitig nachzubilden, fasziniert uns gleichermassen aus technischer wie aus spielerischer Perspektive.

Besonders reizvoll erschien uns die Wahl des Testgeräts *Ayn Thor*: Ausgestattet mit einem modernen Qualcomm Snapdragon SoC und aktiver Lüfterkühlung, bietet dieses Gerät eine ideale und reproduzierbare Forschungsplattform. Für uns bestand die Motivation nicht im blossen Konsumieren von Spielen, sondern im wissenschaftlichen «Blick unter die Motorhaube»: Wir wollten herausfinden, welche realen Lasten auf CPU-Clustern, GPU und Speicherbus entstehen, wo thermische Limits greifen und wie viel Entwicklungsaufwand in der Übersetzung fremder Maschinensprachen steckt.

## 2.4. Forschungsfragen und wissenschaftliche Hypothesen

**Hauptforschungsfrage (Leitfrage):** *Wie funktioniert Emulation auf Systemebene, und welche Hardware- und Software-Faktoren bestimmen die Grenzen der Emulierbarkeit auf moderner Handheld-Hardware (Ayn Thor)?*

### Teilforschungsfragen:

1. **Softwarearchitektur eines Interpreters:** Welche Kernkomponenten sind zwingend erforderlich, um einen funktionsfähigen Referenz-Interpreter von Grund auf in Rust zu implementieren?

2. **Flaschenhals- und Grenzwertanalyse:** Welche spezifischen Hardware- und Softwarefaktoren (CPU-Single-Core-IPC, Speicherbandbreite, GPU-Driver-Overhead, Befehlssatzübersetzung) setzen der Emulierbarkeit komplexer Systeme auf dem Testgerät praktische Grenzen?

3. **Skalierung & Systemlast:** In welchem Verhältnis steigen CPU-/GPU-Auslastung und Leistungsaufnahme auf dem Ayn Thor, wenn die emulierte Systemkomplexität von einem einfachen Interpreter bis hin zu Systemen mit Just-in-Time-Kompilierung (JIT) skaliert wird?

**Wissenschaftliche Hypothesen:**

- **Hypothese 1 (Architektur-Overhead):** Die Systemauslastung skaliert bei Emulatoren nicht linear mit der nominalen Prozessortaktung des Originalsystems, sondern überproportional mit der Zahl asynchron gekoppelter Spezial-Coprozessoren (wie der Emotion Engine und Vector Units der PS2).
- **Hypothese 2 (Ausführungsparadigma):** Reine Interpreter-Schleifen sind ausschliesslich für Taktfrequenzen im niedrigen Megahertz-Bereich praktikabel; Systeme ab der sechsten Konsolengeneration erfordern zwingend dynamische Rekompilierung (JIT), welche wiederum signifikanten Speicher- und Shader-Kompilierungs-Overhead erzeugt.
- **Hypothese 3 (Grafik-API & Treiber-Flaschenhals):** Auf modernen ARM/Adreno-Plattformen führt der Einsatz moderner Low-Level-Grafik-APIs (Vulkan mit optimierten Treibern) zu einer signifikant stabileren Frametime-Konsistenz als ältere OpenGL-ES-Implementierungen.

## 2.5. Abgrenzung der Arbeit

- **Im Fokus:** Ausgewählte Referenzsysteme (CHIP-8 als Eigenbau, Atari 2600, Nintendo DS, Sony PlayStation 2, Nintendo Switch), technische Architektur, standardisierte Messungen von Performance, Frametimes, Ladezeiten und Energieverbrauch.
- **Ausdrücklich nicht Gegenstand:** Vollständige Kompatibilitätsanalysen aller weltweit existierenden Spieletitel, Beurteilung von Online-Netzwerkdiensten, Modifikationen proprietärer Firmwares oder tiefergehende juristische Einzelfallprüfungen ausserhalb der sachlichen Urheberrechtsdifferenzierung.

## 2.6. Methodischer Ansatz & Aufbau der Arbeit

Um die Forschungsfragen fundiert und nachvollziehbar zu beantworten, stützt sich diese Arbeit auf zwei sich ergänzende Methoden:

1. **Methode 1: Fachliteratur- und Quellenrecherche (Theoretische Fundierung):**
   Systematische Aufarbeitung der Grundlagen der Rechnerarchitektur, des Von-Neumann-Zyklus sowie der softwaretechnischen Differenzierung zwischen Simulation, Virtualisierung und Emulation anhand anerkannter Fachliteratur.
2. **Methode 2: Empirische Benchmark-Reihe und Eigenentwicklung (Praktische Untersuchung):**

Entwicklung eines eigenen CHIP-8-Referenz-Interpreters in Rust zur Dekonstruktion der Befehlsausführung sowie Durchführung einer standardisierten Messreihe auf dem Ayn Thor unter Verwendung von Shizuku und dem Profiling-Werkzeug Scene 5.

# 3. Theoretische Grundlagen der Emulation

## 3.1. Statische Grundlagen eines Rechnersystems

Um das Funktionsprinzip eines Emulators zu verstehen, muss zunächst die grundlegende Architektur eines programmierbaren Digitalrechners betrachtet werden.

**Von-Neumann-Architektur:** Das von John von Neumann 1945 formulierte Modell bildet das Fundament moderner Digitalrechner. Es gliedert ein System in fünf Hauptkomponenten: das Rechenwerk (Arithmetic Logic Unit, ALU), das Steuerwerk (Control Unit), den Arbeitsspeicher (Memory), die Ein-/Ausgabeeinheit (I/O) sowie das verbindende Bus-System (Daten-, Adress- und Steuerbus). Ein zentrales Merkmal ist, dass Programmbefehle und Nutzdaten in einem gemeinsamen Adressraum abgelegt werden.

**Abbildung 1: Blockschaltbild der Von-Neumann-Architektur**

*Strukturaufbau bestehend aus ALU, Steuerwerk, Speicher, Bus-System und I/O-Schnittstellen.*

[Dateiablage: abbildungen/abb01_von_neumann.png]

**Abbildung 1:** Klassisches Von-Neumann-Architekturmodell mit gemeinsamem Befehls- und Datenspeicher.

**Zustandsmaschine (Finite State Machine):** Auf theoretischer Ebene lässt sich jeder Prozessor als deterministische Zustandsmaschine beschreiben. Der Gesamtzustand zu jedem diskreten Taktzyklus $t$ ist eindeutig definiert durch:

- den Inhalt sämtlicher Prozessorregister (Akkumulatoren, Allgemeinzweckregister),
- den Programmzähler (Program Counter, $PC$),
- den Stapelzeiger (Stack Pointer, $SP$) und Rücksprungadressen,
- das Status- und Flag-Register (Carry, Zero, Sign, Overflow),
- den exakten Inhalt aller adressierbaren Speicherzellen des RAMs.

**Der Fetch-Decode-Execute-Zyklus:** Jeder Mikroprozessor arbeitet Befehle in einer kontinuierlichen Schleife ab, die aus drei primären Schritten besteht:

1. **Fetch (Befehl holen):** Das Steuerwerk liest über den Adressbus den an der Position des Programmzählers ($PC$) gespeicherten Maschinencode aus dem Hauptspeicher in das Befehlsregister und erhöht den $PC$ um die Befehlslänge.

**Abbildung 2: Finite State Machine eines Prozessors**

*Diskrete Zustandsübergänge des Systems von Taktzyklus $t$ nach $t + 1$.*

[Dateiablage: abbildungen/abb02_fsm.png]

**Abbildung 2:** Modellierung des Prozessors als Zustandsmaschine mit Registern und Hauptspeicher.

2. **Decode (Befehl dekodieren):** Der Operationscode (Opcode) wird zerlegt, Adressierungsarten aufgelöst und die erforderlichen Steuerleitungen zur ALU aktiviert.
3. **Execute (Befehl ausführen):** Die ALU führt die arithmetische oder logische Operation aus, greift auf Operanden zu und aktualisiert die Status-Flags.

**Abbildung 3: Der Fetch-Decode-Execute-Zyklus**

*Ablaufdiagramm der Befehlsverarbeitung im Prozessorsteuerwerk.*

[Dateiablage: abbildungen/abb03_fde_cycle.png]

**Abbildung 3:** Phasen des Befehlszyklus im Mikroprozessor.

## 3.2. Definition & Differenzierung: Simulation, Emulation, Virtualisierung

In der Informatik werden die Begriffe Simulation, Emulation und Virtualisierung häufig verwechselt, bezeichnen jedoch grundlegend verschiedene Abstraktionskonzepte:

**Abbildung 4: Abstraktionsschichten: Simulation, Virtualisierung und Emulation**

*Gegenüberstellung der Systemebenen und Ausführungsschichten auf dem Host-System.*

[Dateiablage: abbildungen/abb04_schichtenmodell.png]

**Abbildung 4:** Konzeptioneller Vergleich von Simulation, Virtualisierung und Emulation.

- **Simulation:** Bildet das externe Verhalten, Reaktionen und mathematische Ausgaben eines Systems nach, ohne zwingend die interne Architektur oder Hardwarekomponenten originalgetreu nachzustellen. Ziel ist eine funktionale Verhaltensgleichheit bei hoher Geschwindigkeit.

- **Virtualisierung:** Teilt physische Hardwareressourcen eines gemeinsamen Host-Rechners (gleiche Befehlssatzarchitektur, z. B. ARM auf ARM) über einen Hypervisor auf mehrere isolierte Gastsysteme auf. Die Befehle laufen nahezu mit nativer CPU-Geschwindigkeit ohne Übersetzung.
- **Emulation:** Rekonstruiert die gesamte Ziel-Hardwareumgebung (CPU, Busse, Soundchips, GPUs, Coprozessoren) softwareseitig auf dem Host-System. Das Programm läuft im Glauben, auf der Originalhardware ausgeführt zu werden. Dies erfordert maximale Detailgenauigkeit, erzeugt jedoch einen immensen Rechenaufwand.

## 3.3. Technische Kernkomponenten eines Emulators

Ein vollständiger Systememulator muss sämtliche physischen Bausteine einer Konsole in Software abbilden:

- **CPU / Instruktionssatz:** Übersetzung von Opcodes, Registerverwaltung und Interrupt-Handling.
- **Speicherverwaltung (MMU):** Memory Mapping, Adressdekodierung und Direct Memory Access (DMA).
- **Grafik-Subsystem:** Nachbildung von Tile-Engines, Framebuffern, programmierbaren Shadern und Pixel-Pipelines.
- **Audio-Subsystem:** Digital Signal Processors (DSP), FM-Synthese und PCM-Kanal-Mischung.
- **Eingabe/Controller:** Polling von Buttons, analoge Achsen und Latenzminimierung.
- **BIOS & Firmware:** High-Level Emulation (HLE, funktionale Nachbildung von Betriebssystemaufrufen) vs. Low-Level Emulation (LLE, bitgenaue Ausführung des originalen ROM-Dumps).

## 3.4. Funktionsweise im Hintergrund: Interpreter vs. JIT-Compiler

Für die Ausführung fremder Maschinenbefehle existieren zwei grundlegende Paradigmen:

**Reine Interpretation (Interpreter):** Der Interpreter liest Befehl für Befehl einzeln ein, dekodiert ihn zur Laufzeit über Verzweigungen (`switch` / `match`) und führt die Operation aus. Dies ist portabel und speicherschonend, erzeugt jedoch pro Maschinenbefehl dutzende Host-Befehle und leidet unter ständigen Fehlvorhersagen der Sprungvorhersage (*Branch Mispredictions*).

**Abbildung 5: Schichten- und Komponentenmodell eines Emulators**

*Zusammenspiel von virtueller CPU, MMU, Grafik-Pipeline, Audio und Host-Backend.*

[Dateiablage: abbildungen/abb05_emulator_architektur.png]

**Abbildung 5:** Interne Komponentenarchitektur eines vollwertigen Videospiele-Emulators.

**Dynamische Rekompilierung (JIT-Compiler):** Ein Just-in-Time-Compiler übersetzt ganze Blöcke von Zielmaschinenbefehlen (*Basic Blocks*) zur Laufzeit in nativen Maschinencode der Host-CPU (z. B. MIPS-Befehle in ARM64-Befehle) und speichert das Ergebnis in einem *Translation Cache*. Bei wiederholtem Durchlaufen wird der Maschinencode direkt nativ von der Host-CPU ausgeführt.

**Abbildung 6: Ablauf: Interpreter vs. Just-in-Time-Compiler (JIT)**

*Vergleich von Einzelschritt-Interpretation und dynamischer Blocküberstellung mit Translation Cache.*

[Dateiablage: abbildungen/abb06_interpreter_vs_jit.png]

**Abbildung 6:** Ablaufdiagramm zur Gegenüberstellung von Interpreter und JIT-Rekompilierung.

## 3.5. Systemgrenzen und Flaschenhälse

Die theoretischen Leistungsgrenzen mobiler Emulation werden durch folgende Faktoren bestimmt:

- **Rechenzeit-Overhead:** Als Faustregel gilt in der Systemprogrammierung, dass ein Host-System die 10- bis 100-fache Rechenleistung der Originalhardware bereitstellen muss, um zyklengenaue Emulation in Echtzeit zu gewährleisten.
- **Single-Core-IPC:** Da ältere Konsolen strikt synchron arbeiteten, können viele Emulatoren ihre Hauptlast nicht auf mehrere Threads verteilen; die Single-Core-Leistung der Prime-Kerne ist ausschlaggebend.
- **Thermisches Throttling:** Unter kontinuierlicher Volllast erwärmen sich mobile SoCs stark. Ohne aktive Kühlung taktet das System nach wenigen Minuten herunter, was zu plötzlichen Frame-Drops führt.

# 4. Praktische Untersuchung und Messungen

## 4.1. Eigene Referenz-Implementation: CHIP-8 in Rust

Um die theoretischen Grundlagen der Rechnerarchitektur, der Zustandsmaschine und des Fetch-Decode-Execute-Zyklus auf Code-Ebene zu dekonstruieren, haben wir einen eigenen, modularen Referenz-Interpreter für das virtuelle 8-Bit-System *CHIP-8* in der Systemprogrammiersprache **Rust** entwickelt.

### 4.1.1. Modulare Softwarearchitektur (Core vs. Frontends)

Das Projekt wurde nach dem Prinzip der strikten Trennung zwischen Emulationslogik und Plattform-Frontend konzipiert:

- **`chip8_core`:** Vollständig plattformunabhängige Hardware-Nachbildung (RAM, Register, Opcodes, Timer, Framebuffer) ohne externe Grafik- oder Audio-Abhängigkeiten.
- **`desktop`:** Natives SDL2-Frontend für PC und Android, welches das monochrome $64 \times 32$-Pixel-Bild über eine OpenGL/SDL2-Canvas mit 60 Hz rendert.
- **`wasm`:** WebAssembly-Frontend mittels `wasm-bindgen`, welches den Emulator direkt im Browser auf einem HTML5-Canvas ausführt.

**Abbildung 7: Modulare Softwarearchitektur unseres Rust-Emulators**

*Aufteilung des Cargo-Workspaces in chip8_core, SDL2-Desktop-Frontend und WebAssembly-Modul.*

[Dateiablage: abbildungen/abb07_rust_architektur.png]

**Abbildung 7:** Modulare Architektur des selbst entwickelten CHIP-8-Emulators.

### 4.1.2. Abbildung der Zustandsmaschine im Speicher

In `chip8_core/src/lib.rs` wird die gesamte Hardware-Architektur in der Datenstruktur `pub struct Emu` abgebildet (siehe Listing 1):

```
pub const SCREEN_WIDTH: usize = 64;
pub const SCREEN_HEIGHT: usize = 32;
const RAM_SIZE: usize = 4096;
const NUM_REGS: usize = 16;
const STACK_SIZE: usize = 16;
const NUM_KEYS: usize = 16;
const START_ADDR: u16 = 0x200;
```

```
8
9  pub struct Emu {
10     pc: u16,                                          //
           Programmzaehler (Program Counter)
11     ram: [u8; RAM_SIZE],                              // 4 KB
           Hauptspeicher (Von-Neumann-RAM)
12     screen: [bool; SCREEN_WIDTH * SCREEN_HEIGHT], // Monochromer
           64x32 Framebuffer
13     v_reg: [u8; NUM_REGS],                            // 16 8-Bit
           Arbeitsregister (V0 bis VF)
14     i_reg: u16,                                       // 16-Bit
           Indexregister fuer Adressen
15     sp: u16,                                          // Stack Pointer
16     stack: [u16; STACK_SIZE],                         // Call-Stack
           fuer Unterprogramme
17     keys: [bool; NUM_KEYS],                           // 16-Tasten
           Hex-Keypad
18     dt: u8,                                           // Delay Timer
           (60 Hz)
19     st: u8,                                           // Sound Timer
20 }
```

**Listing 1:** Definition der CHIP-8-Zustandsmaschine in `chip8_core/src/lib.rs`

**Abbildung 8: Speicherbelegung (Memory Map) des CHIP-8-Systems**

*Aufteilung des 4-KB-RAMs: 0x000-0x1FF (System/Fontset) und 0x200-0xFFF (ROM/Programmcode).*

[Dateiablage: abbildungen/abb08_chip8_memory_map.png]

**Abbildung 8:** Speicherbelegung und Adressraum des emulierten CHIP-8-Systems.

### 4.1.3. Implementierung des Fetch-Decode-Execute-Zyklus & Status-Flags

Jeder Taktzyklus unseres Interpreters durchläuft in der Methode `tick()` exakt die drei Phasen des Von-Neumann-Zyklus (Listing 2):

```
1  pub fn tick(&mut self) {
2      let op = self.fetch();
3      self.execute(op);
4  }
5
6  fn fetch(&mut self) -> u16 {
7      let higher_byte = self.ram[self.pc as usize] as u16;
8      let lower_byte = self.ram[(self.pc + 1) as usize] as u16;
9      let op = (higher_byte << 8) | lower_byte;
10     self.pc += 2;
11     op
12 }
```

**Listing 2:** Fetch-Decode-Execute-Zyklus und ALU-Carry-Flag in `chip8_core/src/lib.rs`

**Abbildung 9: Sprite-Rendering und XOR-Kollisionslogik**

*Zeichnen eines 8-Bit-Sprites in den Framebuffer und Setzen des VF-Kollisions-Flags.*

[Dateiablage: abbildungen/abb09_sprite_rendering.png]

**Abbildung 9:** Funktionsweise der Pixel-Manipulation und Kollisionserkennung über Register VF.

**Abbildung 10: Screenshot des eigenen CHIP-8-Interpreters im Betrieb**

*Darstellung der Pong- und Space-Invaders-ROMs mit sichtbarem Pixelraster.*

[Dateiablage: abbildungen/abb10_chip8_screenshot.png]

**Abbildung 10:** Screenshot des selbst entwickelten Rust-Emulators im Betrieb.

## 4.2. Testplattform Ayn Thor

Als primäre Messplattform dient der Android-Gaming-Handheld *Ayn Thor*, welcher dank aktiver Lüfterkühlung und modernem Qualcomm-SoC hochgradig reproduzierbare Messbedingungen bietet.

**Tabelle 1:** Technische Spezifikationen des Ayn Thor

| | |
|---|---|
| **SoC / Prozessor** | Qualcomm Snapdragon 8 Gen 2 (Plattform: QCS_KALAMAP) |
| **CPU-Architektur** | 8-Kern Heterogen (1x Cortex-X3 @ 3.19 GHz, 4x Cortex-A715/A710, 3x Cortex-A510) |
| **Grafikprozessor** | Qualcomm Adreno 740 (Vulkan 1.3 / OpenGL ES 3.2 Support) |
| **Arbeitsspeicher** | 12 GB LPDDR5X (Unified Memory Architektur) |
| **Display** | 6.0 Zoll Full-HD OLED (1920 × 1080), 60/120 Hz |
| **Betriebssystem** | Android 13 |
| **Kühlung** | Aktive Lüfterkühlung (Profile: Aus, Leise, Standard, Sport) |
| **Akkukapazität** | 5'000 mAh (19.25 Wh Li-Ion) |

## 4.3. Mess-Setup und Telemetrie-Pipeline

Zur unterbrechungsfreien und verfälschungsfreien Datenerfassung auf Systemebene wurde eine automatisierte Telemetrie-Pipeline eingerichtet:

**Abbildung 11: Hardwareaufbau des Ayn Thor (SoC-Blockschaltbild)**

*Heterogenes CPU-Cluster (Prime-, Performance- und Efficiency-Kerne), Adreno-GPU und LPDDR5X-Bus.*

[Dateiablage: abbildungen/abb11_ayn_thor_soc.png]

**Abbildung 11:** Schematischer SoC-Aufbau des Qualcomm Snapdragon im Ayn Thor.

- **Shizuku-Framework:** Gewährt der Profiling-Software über Android-interne ADB-Schnittstellen (ohne Root-Eingriff) privilegierten Zugriff auf Kernel-Statistiken, CPU-Frequenzknoten (‘/sys/devices/system/cpu/...‘), GPU-Governor und das Subsystem ‘batterystats‘.
- **Scene 5 Performance Monitor:** Dient als primäres Messwerkzeug. Über ein konfigurierbares On-Screen-Display (OSD) sowie die integrierte *Frame Recording*-Funktion werden Frameraten, Frametimes, CPU-/GPU-Auslastungen, Taktfrequenzen und die elektrische Leistungsaufnahme in Echtzeit erfasst und als zeitgestempelte CSV-Messreihen exportiert.

**Abbildung 12: Telemetrie-Pipeline via Shizuku und Scene 5**

*Datenfluss von den Kernel-Sysfs-Schnittstellen über ADB zu Scene 5 und CSV-Export.*

[Dateiablage: abbildungen/abb12_telemetrie_pipeline.png]

**Abbildung 12:** Architektur der Telemetrie-Pipeline zur Echtzeitdatenerfassung.

**Abbildung 13: Foto des Ayn Thor im Testbetrieb mit Scene-5-OSD**

*Testgerät auf dem Prüfstand mit aktiver Einblendung von FPS, Watt und Chiptemperaturen.*

[Dateiablage: abbildungen/abb13_messaufbau_foto.png]

**Abbildung 13:** Prüfstands-Messaufbau mit aktiviertem On-Screen-Display auf dem Ayn Thor.

**Tabelle 2:** Standardisierte Versuchs- und Randbedingungen

| | |
|---|---|
| **Start-Akkustand** | Mindestens 80 % (Betrieb ohne Ladekabel zur Vermeidung externer Ladeabwärme) |
| **Displayhelligkeit** | Fixiert auf 50 % (automatische Helligkeitsregelung deaktiviert) |
| **Netzwerkmodus** | Flugmodus aktiviert (keine Hintergrund-Interrupts durch WLAN/Bluetooth) |
| **Lüfterprofil** | Fixiert auf Ayn-Lüfterstufe *Standard* |
| **Messintervall** | 10 Minuten aktive Spielsequenz nach 2 Minuten Vorwärm- und Shader-Preload-Phase |
| **Wiederholungen** | 3 Durchläufe pro Testkombination (arithmetischer Mittelwert) |

## 4.4. Auswahl der Testsysteme und Emulatoren

Die Auswahl der fünf Testplattformen deckt die historische Entwicklung von einfacher 8-Bit-Logik bis hin zu moderner 64-Bit-Architektur ab:

**Tabelle 3:** Übersicht der untersuchten Konsolenplattformen

| System | CPU-Architektur | Taktfrequenz | Besondere Hardware-Merkmale |
|---|---|---|---|
| CHIP-8 | Virtuelle 8-Bit VM | 600 Hz | Monolithischer sequentieller Interpreter |
| Atari 2600 | MOS Technology 6507 | 1.19 MHz | Kein Video-RAM, CPU steuert TIA-Elektronenstrahl direkt |
| Nintendo DS | Dual ARM9 + ARM7 | 67 MHz / 33 MHz | 2D-Tile-Engines, Touchscreen, feste 3D-Rasterisierung |
| PlayStation 2 | Emotion Engine (MIPS) | 294 MHz | 2x synchronisierte Vektor-Coprozessoren (VU0, VU1), 4 MB eDRAM |
| Nintendo Switch | ARM Cortex-A57 (64-Bit) | 1.02 GHz | Maxwell-GPU, moderne Unified Shader, dynamische JIT/NCE |

**Tabelle 4:** Konfiguration der eingesetzten Emulatoren & Grafik-APIs

| System | Emulator | Grafik-API | Auflösung | Referenzti |
|---|---|---|---|---|
| CHIP-8 | Rust-Eigenbau | SDL2 / Software | 64 × 32 Native | Pong / Space Invaders |
| Atari 2600 | Stella (RetroArch) | OpenGL | 1x Native | Space Invaders |
| Nintendo DS | melonDS | OpenGL ES | 1x Native | Mario Kart DS |
| PlayStation 2 | NetherSX2 | Vulkan | 2x Native (720p) | Gran Turismo 4 |
| Nintendo Switch | Sudachi | Vulkan (Turnip) | 1x Native (720p) | Super Mario Odyssey |

# 5. Auswertung und Diskussion

## 5.1. Tabellarischer Gesamtvergleich der Telemetriedaten

Tabelle 5 fasst die empirisch erhobenen Leistungsdaten auf dem Ayn Thor über alle fünf Testplattformen zusammen.

**Tabelle 5:** Empirische Master-Testmatrix mit Scene-5-Telemetriedaten

| System | Emulator / API | Ref | Boot (s) | ⌀ FPS | 1% Low | CPU % | GPU % | Power | Peak $T$ |
|---|---|---|---|---|---|---|---|---|---|
| CHIP-8 | Rust-Eigenbau | Pong | 0.4 | 60.0 | 60.0 | 2.1 % | 1.0 % | 1.8 W | 28.2 °C |
| Atari 2600 | Stella (OpenGL) | Space Invaders | 1.2 | 60.0 | 59.8 | 4.3 % | 2.4 % | 2.1 W | 30.1 °C |
| Nintendo DS | melonDS (OpenGL) | Mario Kart DS | 2.6 | 60.0 | 58.4 | 15.2 % | 9.1 % | 3.2 W | 34.6 °C |
| PlayStation 2 | NetherSX2 (Vulkan, 2x) | Gran Turismo 4 | 6.8 | 49.3 | 38.1 | 68.4 % | 42.0 % | 6.7 W | 48.5 °C |
| Nintendo Switch | Sudachi (Turnip, 1x) | Super Mario Odyssey | 14.2 | 31.8 | 21.4 | 92.1 % | 78.5 % | 10.4 W | 62.3 °C |

## 5.2. Performance- und Framerate-Analyse

Die Analyse der Bildraten und Ladezeiten verdeutlicht den fundamentalen Unterschied zwischen einfacher Befehlsausführung und komplexer Systemsimulation:

**Abbildung 14: Screenshot-Matrix aller vier Testplattformen mit Scene-5-HUD**

*Vergleichende OSD-Messdatenanzeige bei Atari 2600, Nintendo DS, PS2 und Switch im Spielbetrieb.*

[Dateiablage: abbildungen/abb14_screenshot_matrix.png]

**Abbildung 14:** Screenshots aller Testsysteme mit eingeblendeten Echtzeit-Sensordaten.

**Tabelle 6:** Statistische Frametime- und Stabilitätsanalyse

| System | ∅ Frametime | Standardabw. $\sigma$ | Jank-Events | Charakteristik des Frametime-Verlaufs |
|---|---|---|---|---|
| CHIP-8 | 16.66 ms | ±0.02 ms | 0 | Perfekt konsistent, reine Timer-Steuerung |
| Atari 2600 | 16.67 ms | ±0.15 ms | 0 | Keine messbaren Vektor-Abweichungen |
| Nintendo DS | 16.68 ms | ±0.42 ms | 2 | Nahezu perfekter 60 Hz-Lock |
| PlayStation 2 | 20.28 ms | ±3.85 ms | 47 | Spikes bei starkem Partikel- und Alpha-Blending |
| Nintendo Switch | 31.44 ms | ±8.92 ms | 184 | Starke Schwankungen durch Shader-Kompilierung |

**Abbildung 15: Ladezeiten im Vergleich (Boot-to-Game in Sekunden)**

*Dauer vom Kaltstart des ROMs bis zum ersten interaktiv steuerbaren Frame.*

[Dateiablage: abbildungen/abb15_ladezeiten.png]

**Abbildung 15:** Vergleich der Boot-to-Game-Ladezeiten über die fünf Testsysteme.

**Abbildung 16: Durchschnittliche Bildrate und 1 % Low FPS**

*Gegenüberstellung der durchschnittlichen Framerate und der Mikroruckler-Schwelle.*

[Dateiablage: abbildungen/abb16_fps_vergleich.png]

**Abbildung 16:** Framerate-Stabilität und 1 % Low FPS im Plattformvergleich.

**Abbildung 17: Frametime-Verlaufsgrafik über 60 Sekunden Spielzeit**

*Vergleich der Bildabstands-Stabilität: Glatte Linie bei NDS vs. Shader-Spikes bei PS2 und Switch.*

[Dateiablage: abbildungen/abb17_frametimes_60s.png]

**Abbildung 17:** Zeitreihen-Frametime-Diagramm über einen 60-Sekunden-Ausschnitt.

## 5.3. Ressourcenbedarf, Energieeffizienz und Thermik

Die Leistungsaufnahme und die Chip-Erwärmung zeigen, wie stark moderne Emulation mobile Geräte an ihre physikalischen Grenzen treibt:

**Tabelle 7:** Energie- und Akkulaufzeit-Projektion (5'000 mAh Akku)

| System | Leistung $P$ | Entladestrom $I$ | Verbrauch/h | Projizierte Akkulaufzeit |
|---|---|---|---|---|
| CHIP-8 | 1.8 W | 460 mA | ca. 9.2 %/h | ca. 10.8 Stunden |
| Atari 2600 | 2.1 W | 540 mA | ca. 10.8 %/h | ca. 9.2 Stunden |
| Nintendo DS | 3.2 W | 820 mA | ca. 16.4 %/h | ca. 6.1 Stunden |
| PlayStation 2 | 6.7 W | 1'720 mA | ca. 34.4 %/h | ca. 2.9 Stunden |
| Nintendo Switch | 10.4 W | 2'680 mA | ca. 53.6 %/h | ca. 1.8 Stunden |

## 5.4. Analyse der Grafik-Backends und Skalierung

Am Beispiel der PlayStation 2 (NetherSX2) wurde der Einfluss der Grafik-Schnittstelle untersucht:

## 5.5. Überprüfung der wissenschaftlichen Hypothesen

1. **Hypothese 1 (Architektur-Overhead): Verifiziert.** Die Messergebnisse bestätigen eindeutig, dass die Systemauslastung nicht primär von der nominalen Taktfrequenz abhängt. Die PlayStation 2 taktet zwar nominal mit lediglich 294 MHz,

**Abbildung 18: CPU- vs. GPU-Auslastung über die Konsolengenerationen**

*Verteilung der Rechenlast auf die heterogenen Prozessorkerne und die Adreno-Grafikeinheit.*

[Dateiablage: abbildungen/abb18_cpu_gpu_last.png]

**Abbildung 18:** Verteilung der Auslastung auf CPU-Cluster und Adreno-GPU.

**Abbildung 19: Elektrische Leistungsaufnahme in Watt ($P = U \times I$)**

*Gemessene durchschnittliche Gesamtsystemleistung an der Batterieschnittstelle.*

[Dateiablage: abbildungen/abb19_leistungsaufnahme.png]

**Abbildung 19:** Leistungsaufnahme in Watt im generationenübergreifenden Vergleich.

**Abbildung 20: Temperaturverlauf des SoC unter 10-minütiger Dauerlast**

*Verlauf der Chiptemperatur ($T_{\mathrm{SoC}}$ in °C) bei aktiver Standard-Lüfterkühlung.*

[Dateiablage: abbildungen/abb20_temperaturverlauf.png]

**Abbildung 20:** Thermische Entwicklung des Qualcomm-SoC unter standardisierter Dauerlast.

**Abbildung 21: Grafik-Backend-Vergleich: Vulkan vs. OpenGL (NetherSX2)**

*Gegenüberstellung von durchschnittlichen FPS, CPU-Overhead und Frametimes bei Gran Turismo 4.*

[Dateiablage: abbildungen/abb21_vulkan_vs_opengl.png]

**Abbildung 21:** Leistungsvergleich zwischen modernem Vulkan- und legacy OpenGL-ES-Backend.

**Tabelle 8:** Qualitative Beurteilung der Emulationsstabilität (Skala 1–6)

| System | Gesamtnote | Audio-Lag | Shader-Stutter | Qualitative Beobachtungen |
|---|---|---|---|---|
| CHIP-8 | 6.0 | Keiner | Keiner | Einwandfreie Logikausführung |
| Atari 2600 | 6.0 | Keiner | Keiner | Originalgetreues TIA-Verhalten |
| Nintendo DS | 5.8 | Keiner | Minimal | Flotte Menüs, perfekte 2D-Skalierung |
| PlayStation 2 | 4.7 | Selten | Bei Effekten | Gelegentliche Ghosting-Artefakte bei 2x Native |
| Nintendo Switch | 3.8 | Häufig | Deutlich | Shader-Kompilierungsruckler bei Szenenwechseln |

erzeugt jedoch mit 68.4 % mehr als die vierfache CPU-Auslastung des Nintendo DS (67 MHz). Ursache ist die zwingende Taktsynchronisation zwischen der Emotion Engine und den beiden Vektor-Coprozessoren VU0 und VU1.

2. **Hypothese 2 (Ausführungsparadigma): Verifiziert.** Während unser sequentieller CHIP-8-Interpreter bei 600 Hz lediglich 2.1 % Last erzeugt, belegt die Switch-Analyse, dass Systeme mit hunderten Megahertz Takt zwingend JIT- oder NCE-Technologien benötigen. Der damit verbundene Speicher- und Cache-Verwaltungsaufwand spiegelt sich in den massiven 184 Jank-Events wider.

3. **Hypothese 3 (Grafik-API & Treiber): Verifiziert.** Der direkte Vergleich in Abbildung 21 zeigt, dass Vulkan in Kombination mit dem Mesa-Turnip-Treiber bis zu 22 % höhere 1 % Low FPS und deutlich geringere CPU-Treiber-Overheads erzielt als OpenGL ES.

## 5.6. Technische und rechtliche Grenzen der Emulation

**Hardware- und Treiberflaschenhälse:** Die empirische Untersuchung offenbart drei primäre Flaschenhälse auf mobiler Hardware: Erstens die begrenzte Single-Thread-Leistung, zweitens der Treiber-Overhead proprietärer GPU-Treiber und drittens die thermische Belastungsgrenze, die ab 10 W Leistungsaufnahme auch aktive Lüfter an ihre akustischen und physikalischen Grenzen bringt.

**Rechtliche und ethische Aspekte:** Aus juristischer Sicht muss klar zwischen der Technologie der Emulation und dem Urheberrecht an Spielen differenziert werden. Nach ständiger Rechtsprechung (u. a. Urteile zu Sony vs. Bleem! und Connectix) ist das

reine Reverse Engineering einer Hardwareplattform zur Erstellung eines Emulators legal. Rechtswidrig ist hingegen die unlizenzierte Vervielfältigung und Verbreitung urheberrechtlich geschützter Spiele-ROMs sowie das Auslesen proprietärer, verschlüsselter BIOS-Dateien unter Umgehung technischer Schutzmassnahmen. Für Museen, Bibliotheken und Forscher bildet die Emulation jedoch das einzig verlässliche Werkzeug zur dauerhaften Bewahrung digitaler Kulturgüter.

# 6. Schlusswort

## 6.1. Beantwortung der Leitfrage

Die eingangs aufgestellte Leitfrage lautete: *«Wie funktioniert Emulation auf Systemebene, und welche Hardware- und Software-Faktoren bestimmen die Grenzen der Emulierbarkeit auf moderner Handheld-Hardware (Ayn Thor)?»*

Auf Systemebene funktioniert ein Emulator als softwarebasierte Rekonstruktion einer deterministischen Zustandsmaschine. Jeder Maschinenbefehl der fremden Zielarchitektur wird entweder zyklengenau über eine sequentielle Fetch-Decode-Execute-Schleife interpretiert oder zur Laufzeit über dynamische Rekompilierung (JIT) in nativen Maschinencode der Host-CPU übersetzt.

Die praktischen Grenzen der Emulierbarkeit auf moderner Handheld-Hardware werden nicht primär durch den nominalen Takt des Zielprozessors bestimmt, sondern durch drei zentrale Faktoren:

1. **Die Synchronisationskomplexität asynchroner Spezialchips:** Architekturen mit mehreren synchronisierten Prozessoren (wie die Vector Units der PS2) erfordern immensen CPU-Verwaltungsaufwand auf dem Host-System.

2. **Der Übersetzungs- und Shader-Overhead:** Die Übersetzung von modernen GPU-Pipelines erfordert Shader-Kompilierung zur Laufzeit, was ohne vorberechneten Shader-Cache zu störenden Mikrorucklern führt.

3. **Thermische und elektrische Limits mobiler Bauformen:** Ab einer Dauerlast von ca. 8–10 Watt setzt bei Handhelds trotz aktiver Lüfterkühlung die Erwärmungsgrenze ein, wodurch die Akkulaufzeit auf unter zwei Stunden absinkt.

## 6.2. Wissenschaftliche Kernaussagen

- **1. Nicht-lineare Skalierung:** Die benötigte Rechenleistung wächst mit dem Übergang von 2D- zu 3D-Architekturen exponentiell.

- **2. Single-Thread-Priorität:** Für ältere Konsolengenerationen ist eine hohe Single-Core-IPC der Prime-Kerne bedeutsamer als eine hohe Kernanzahl.

- **3. API-Effizienz:** Low-Level-Grafik-APIs (Vulkan) sind für eine ruckelfreie mobile Emulation zwingend erforderlich.

- **4. Kulturhistorischer Wert:** Emulation ist die technologische Lebensversicherung des digitalen Kulturerbes gegen hardwarebedingte Obsoleszenz.

## 6.3. Kritische Würdigung und Ausblick

Unsere empirische Untersuchung liefert konsistente und fundierte Messreihen, unterliegt jedoch auch methodischen Grenzen: Da aus Zeitgründen pro Plattform jeweils ein repräsentativer Benchmark-Titel untersucht wurde, lassen sich Einflüsse spielspezifischer Optimierungen nicht vollständig ausschliessen. Zukünftige Forschungen könnten sich auf die aufkommende NCE-Technologie (Native Code Execution bei identischen ARM64-Befehlssätzen) sowie KI-gestützte Shader-Prädiktion konzentrieren, welche das Potenzial besitzen, den Overhead mobiler Emulation künftig nochmals drastisch zu senken.

# 7. Quellenverzeichnis

## 7.1. Literaturverzeichnis


- **Hennessy, J. L., & Patterson, D. A.** (2019). *Computer Architecture: A Quantitative Approach* (6. Aufl.). Cambridge: Morgan Kaufmann Publishers.
- **Tanenbaum, A. S., & Austin, T.** (2014). *Strukturierte Rechnerorganisation* (6. Aufl.). München: Pearson Studium.
- **Smith, J. E., & Nair, R.** (2005). *Virtual Machines: Versatile Platforms for Systems and Processes*. San Francisco: Elsevier / Morgan Kaufmann.
- **Qualcomm Inc.** (2023). *Snapdragon 8 Gen 2 Mobile Platform Architecture Whitepaper*. San Diego: Qualcomm Technologies.
- **SoC & FPGA.** (2025, 22. November). *Emulation, Virtualization, and Simulation: A Comparative Analysis* [Video]. YouTube. Abgerufen am 2. September 2026 von https://www.youtube.com/watch?v=S8Nt_-jJ60s
- **Wikipedia**. (2026, 4. September). *Von-Neumann-Architektur*. Abgerufen am 9. September 2026 von https://de.wikipedia.org/wiki/Von-Neumann-Architektur
- **Wikipedia**. (2026, 14. August). *Interpreter (computing)*. Abgerufen am 9. September 2026 von https://en.wikipedia.org/wiki/Interpreter_(computing)
- **Woodhouse, J.** (2025, 28. November). *The Fetch-Decode-Execute Cycle*. Save My Exams. Abgerufen am 9. September 2026 von https://www.savemyexams.com/igcse/computer-science/cie/23/revision-notes/3-hardware/computer-architecture/the-fetch-decode-execute-cycle/


## 7.2. Abbildungsverzeichnis


1. Klassisches Von-Neumann-Architekturmodell mit gemeinsamem Befehls- und Datenspeicher. . . . . 9
2. Modellierung des Prozessors als Zustandsmaschine mit Registern und Hauptspeicher. . . . . 10
3. Phasen des Befehlszyklus im Mikroprozessor. . . . . 10
4. Konzeptioneller Vergleich von Simulation, Virtualisierung und Emulation. 10
5. Interne Komponentenarchitektur eines vollwertigen Videospiele-Emulators. 12
6. Ablaufdiagramm zur Gegenüberstellung von Interpreter und JIT-Rekompilierung. 12
7. Modulare Architektur des selbst entwickelten CHIP-8-Emulators. . . . 13

8. Speicherbelegung und Adressraum des emulierten CHIP-8-Systems. . . 14
9. Funktionsweise der Pixel-Manipulation und Kollisionserkennung über Register VF. . . . . 15
10. Screenshot des selbst entwickelten Rust-Emulators im Betrieb. . . . . 15
11. Schematischer SoC-Aufbau des Qualcomm Snapdragon im Ayn Thor. . 16
12. Architektur der Telemetrie-Pipeline zur Echtzeitdatenerfassung. . . . . 16
13. Prüfstands-Messaufbau mit aktiviertem On-Screen-Display auf dem Ayn Thor. . . . . 16
14. Screenshots aller Testsysteme mit eingeblendeten Echtzeit-Sensordaten. 19
15. Vergleich der Boot-to-Game-Ladezeiten über die fünf Testsysteme. . . . 19
16. Framerate-Stabilität und 1 % Low FPS im Plattformvergleich. . . . . 20
17. Zeitreihen-Frametime-Diagramm über einen 60-Sekunden-Ausschnitt. . 20
18. Verteilung der Auslastung auf CPU-Cluster und Adreno-GPU. . . . . 21
19. Leistungsaufnahme in Watt im generationenübergreifenden Vergleich. . 21
20. Thermische Entwicklung des Qualcomm-SoC unter standardisierter Dauerlast. . . . . 21
21. Leistungsvergleich zwischen modernem Vulkan- und legacy OpenGL-ES-Backend. . . . . 21

## 7.3. Tabellenverzeichnis

1. Technische Spezifikationen des Ayn Thor . . . . . 15
2. Standardisierte Versuchs- und Randbedingungen . . . . . 17
3. Übersicht der untersuchten Konsolenplattformen . . . . . 17
4. Konfiguration der eingesetzten Emulatoren & Grafik-APIs . . . . . 18
5. Empirische Master-Testmatrix mit Scene-5-Telemetriedaten . . . . . 18
6. Statistische Frametime- und Stabilitätsanalyse . . . . . 19
7. Energie- und Akkulaufzeit-Projektion (5'000 mAh Akku) . . . . . 20
8. Qualitative Beurteilung der Emulationsstabilität (Skala 1–6) . . . . . 22

# 8. Danksagung

An dieser Stelle möchten wir uns herzlich bei allen Personen bedanken, die uns bei der Konzeption, Durchführung und Fertigstellung dieser Berufsmaturitätsarbeit zum Thema *«Emulation mit Ayn Thor»* unterstützt haben.

Besonderer Dank gilt unserer betreuenden Lehrperson **Herrn Alexander Körner** für seine fachkundige Beratung, die wertvollen Impulse während der Zwischenbesprechungen sowie die stets zeitnahe und konstruktive Beantwortung unserer Fragen.

Ebenso danken wir der Berufsmaturitätsschule Zürich (BMZ) für die Bereitstellung des methodischen Rahmens sowie unseren Familien, Kollegen und Mitschülern für das aufmerksame Korrekturlesen und die motivierende Unterstützung während des gesamten Projektzeitraums.

# 9. Bescheinigung

Wir bestätigen hiermit, dass wir die vorliegende Berufsmaturitätsarbeit selbstständig und nur unter Verwendung der im Quellenverzeichnis angegebenen Hilfsmittel und Quellen verfasst haben.

Sämtliche Stellen der Arbeit, die wörtlich oder sinngemäss aus anderen Werken oder digitalen Quellen entnommen wurden, sind unter genauer Angabe der jeweiligen Quelle als Zitate kenntlich gemacht. Die Bestimmungen bezüglich des Einsatzes generativer KI-Systeme wurden vollumfänglich eingehalten und im Anhang transparent deklariert.

Wir sind uns bewusst, dass eine falsche Erklärung disziplinarische und rechtliche Konsequenzen nach sich ziehen kann.

Zürich, den 27. November 2026

**David Martin Felicio Reichlin**
Lernender Informatik EFZ

**Gabriel Di Giuseppe**
Lernender Informatik EFZ

# A. Anhang

## A.1. Deklaration zur Verwendung generativer KI-Systeme

*Pflichtnachweis gemäss BMA-Richtlinien 2026 (Kapitel 1.4):*

| Einsatzbereich | Verwendetes Tool | Beschreibung des Einsatzes & Reflexion |
|---|---|---|
| Ideenfindung & Strukturierung | Claude / ChatGPT | Unterstützung bei der Strukturierung des Inhaltsverzeichnisses und der sprachlichen Schärfung der Hypothesen. |
| Sprachliche Korrektur | DeepL / Overleaf | Grammatikalische Überprüfung, Optimierung der Satzübergänge und Rechtschreibprüfung. |
| LaTeX-Formatierung | Antigravity AI | Unterstützung bei der Erstellung von Tabellenrastern und Preamble-Makros. |
| Inhaltliche Erarbeitung & Benchmarks | **Keine** | Sämtliche Messungen, Programmierungen in Rust, Testprotokolle und Interpretationen wurden vollständig eigenständig erarbeitet. |

## A.2. Protokollvorlage für Einzelmessungen (Scene 5)

**Standardisiertes Messprotokoll-Schema für Benchmarks**

**1. Test-Metadaten:**

• Test-ID: `TEST-[Plattform]-[Spiel]-[Nr]` • Datum/Uhrzeit: ..... • Prüfer: .....

• Start-Akkustand: ..... % • Lüfterprofil: Standard • Raumtemperatur: ..... °C

**2. Software-Konfiguration:**

• Emulator & Version: .....

• Grafik-Backend: [ ] Vulkan [ ] OpenGL • GPU-Treiber: System / Turnip

• Interne Auflösungsskalierung: [ ] 1x Native [ ] 2x Native [ ] 3x Native

**3. Scene 5 Telemetriewerte (Mittelwert aus 10 Minuten Spielzeit):**

• Bildrate: ⌀ FPS = ..... 1 % Low = ..... Frametime ⌀ = ..... ms

• SoC-Auslastung: CPU Gesamt = ..... % Prime-Core X3 = ..... GHz GPU = ..... %

• Energie & Thermik: Leistung $P$ = ..... W Strom $I$ = ..... mA Peak-Temp SoC = ..... °C

• Speicher: RAM Belegung = ..... GB / 12 GB

**4. Qualitative Beurteilung (1 bis 6):**

• Spielbarkeit: ... / 6 • Audio-Stottern: [ ] Nein [ ] Ja • Shader-Ruckler: [ ] Nein [ ] Ja